

Relatório interno de hardening para mutmut

Objetivo

Partimos de aproximadamente 178/248 mutantes mortos e usamos uma sequência de testes de contrato e fronteira para subir o score sem alterar o código de produção. O foco foi cobrir o que ainda escapava em `main.py`, `database.py`, `service.py`, `repository.py` e `rules.py`.

Linha do tempo

Rodada 1: base funcional e cobertura de HTTP

- O primeiro bloco de trabalho atacou `main.py`.
- Foram criados testes de camada HTTP com `TestClient`.
- Como a importação direta do módulo dependia de uma combinação instável de dependências SQLAlchemy/FastAPI no ambiente local, o módulo passou a ser carregado com stubs mínimos de SQLAlchemy.
- Também foi validado o wiring dos providers `get_credit_service` e `get_user_service`, garantindo que o `db` recebido é repassado para o repositório SQL e que o serviço de usuários é inicializado de forma lazy, sem quebrar a coleta do mutmut.

Rodada 2: contrato do repositório SQL

- Em seguida foi tratado `database.py`.
- Criamos um teste com fake ORM e fake DB para validar o contrato do `SqlCreditRepository` sem depender de SQLAlchemy real.
- O fake passou a capturar:
 - kwargs exatos enviados ao `SimulacaoORM`
 - chamadas de `add`, `commit` e `refresh`
 - modelo consultado em `query(...)`
 - parâmetros de `filter(...)`, `order_by(...)` e `limit(...)`
- Essa rodada matou mutantes ligados a campos omitidos, limites de listagem, chamadas do fluxo SQL e persistência de `usuario_id`.

Rodada 3: service, idempotência e default de histórico

- Depois o foco foi `service.py`.
- Foram reforçados testes de histórico e idempotência.
- A validação mais importante foi o default explícito de `listar_historico()` com `limite=50`, garantindo que o service repassa o valor correto ao repositório.
- Também foram testados o limite exato de 60 segundos e o cache de idempotência.

Rodada 4: estado do repositório em memória

- O `InMemoryCreditRepository` passou por endurecimento adicional.
- Os testes de estado passaram a verificar:

- ordem reversa de listagem
 - limite explícito
 - default de 50 registros
 - truncamento real com 51 inserções
- Isso ajudou a matar mutantes de off-by-one no comportamento padrão.

Rodada 5: fluxo de dados e fronteiras de score

- Por fim, os testes de `rules.py` foram ampliados com fronteiras exatas dos scores.
- Além dos cenários já existentes, foram adicionados casos em 801, 601, 401 e 400 para as duas modalidades.
- O objetivo era distinguir mutações em comparações e no arredondamento final da taxa.

Rodada 6: endurecimento de contratos de autenticação e usuários

- Os testes de `main.py` passaram a validar os helpers de autenticação de forma direta.
- Também foram reforçados os testes de `user_service.py` com:
 - erro de login para email inexistente
 - hash de senha com salt
 - lookup de token com hash exato
 - formato e chave de retorno do login
- O repositório em memória de usuários recebeu cobertura direta de criação, sessão e lookup por token.

Como foi validado

- `pytest tests/test_01_equivalence.py tests/test_08_database.py -q`
- `pytest tests/test_06_integration.py -q`
- `pytest tests/test_04_data_flow.py -q`
- `pytest tests/test_07_main.py tests/test_09_users_service.py -q`
- `pytest tests/test_08_database.py tests/test_09_users_service.py tests/test_11_user_linked_simulations_service.py -q`
- `mutmut run`

Resultado obtido

- O score saiu de cerca de 178/248 para 467/490 .
- Os mutantes mortos passaram a cobrir principalmente:
 - defaults e roteamento de autenticação em `main.py`
 - login, hash e lookup de token em `user_service.py`
 - persistência e consulta do repositório SQL
 - histórico e idempotência em `service.py`
 - listagem e truncamento do repositório em memória
 - fronteiras de cálculo de juros

Sobre os sobreviventes restantes

Os 23 mutantes que ficaram vivos após a última rodada foram:

- `credit_engine.database.SqlCreditRepository.buscar_por_hash__mutmut_2`
- `credit_engine.database.SqlCreditRepository.listar_simulacoes__mutmut_1`
- `credit_engine.database.SqlCreditRepository.listar_simulacoes_por_usuario__mutmut_1`
- `credit_engine.user_service.UserService._gerar_hash_senha__mutmut_10`
- `credit_engine.user_service.UserService._gerar_hash_senha__mutmut_11`
- `credit_engine.user_service.UserService.registrar_usuario__mutmut_2`
- `credit_engine.user_service.UserService.registrar_usuario__mutmut_5`
- `credit_engine.user_service.UserService.registrar_usuario__mutmut_8`
- `credit_engine.user_service.UserService.registrar_usuario__mutmut_9`
- `credit_engine.user_service.UserService.registrar_usuario__mutmut_15`
- `credit_engine.user_service.UserService.login__mutmut_5`
- `credit_engine.user_service.UserService.login__mutmut_14`
- `credit_engine.user_service.UserService.login__mutmut_15`
- `credit_engine.user_service.UserService.login__mutmut_16`
- `credit_engine.user_service.UserService.login__mutmut_18`
- `credit_engine.service.CreditService.avaliar_credito__mutmut_37`
- `credit_engine.service.CreditService.avaliar_credito__mutmut_39`
- `credit_engine.service.CreditService.avaliar_credito__mutmut_40`
- `credit_engine.service.CreditService.listar_historico__mutmut_1`
- `credit_engine.repository.InMemoryCreditRepository.listar_simulacoes__mutmut_1`
- `credit_engine.repository.InMemoryCreditRepository.listar_simulacoes_por_usuario__mutmut_1`
- `credit_engine.rules.x_calcular_taxa_juros__mutmut_21`
- `credit_engine.main.x_get_user_service__mutmut_4`

Por que esses foram deixados vivos

- O bloco de `user_service.py` ainda mistura mutações de hash, mensagens de exceção e variações de retorno, algumas bem próximas de equivalentes.
- O bloco de `service.py` ainda tem mutações que alteram campos persistidos ou a forma do histórico, e a cobertura atual já valida bastante comportamento público sem entrar em excesso no detalhe interno.
- O sobrevivente de `rules.py` altera o arredondamento final; com as fronteiras já cobertas, esse caso ficou muito próximo de um mutante de baixo retorno.
- O sobrevivente de `main.py` ainda aponta para o provider `get_user_service`; ele já foi endurecido, mas o mutante remanescente indica uma diferença interna que não altera o comportamento público observado nos testes atuais.

Conclusão prática

- O trabalho útil foi concentrado em testes de contrato e fronteira, não em mexer no código de produção.

- O ganho mais forte veio de reforçar defaults, autenticação, persistência do repositório SQL e fronteiras de score.
- Os sobreviventes finais já estão concentrados em detalhes internos e alguns parecem equivalentes ou pouco rentáveis de atacar sem tornar os testes frágeis demais.